FamilyCheck

A scam-awareness and trusted-contact Android application

PROJECT REPORT

Subject: Mobile App Development

Submitted by
Aravindh P — AM.SC.U4CYS23011
Richu James — AM.SC.U4CYS23036

Report date: 29 October 2026

Project overview

FamilyCheck helps an older adult record a concerning message or call, select warning signs and consult someone they trust. It combines a guided checking workflow, locally saved records, trusted contacts, awareness content and optional follow-up notifications.

The project uses Java, XML and SQLite with standard Android components. It is an educational support tool rather than an automatic scam detector or emergency service.

Contents

1. Introduction and objectives
2. Requirements and functional modules
3. Architecture and workflow
4. Database and implementation
5. Interface and syllabus mapping
6. Testing, privacy and limitations
7. Conclusion and future scope

1. Introduction and objectives

Abstract

FamilyCheck is an Android application intended to help senior citizens respond more carefully to suspicious calls and messages. A guided form captures an interaction and manually selected warning signs. Users can then open a known contact, review a sharing preview and save their own assessment. Searchable history, favourite contacts, awareness cards, an offline quiz and local reminders extend the workflow.

The implementation uses Java, XML, Activities, a Fragment, native Android widgets and SQLite. External messaging and dialer applications handle communication through Intents. The app neither monitors communications nor automatically classifies messages as safe or fraudulent.

Problem statement

An unexpected request can create uncertainty about identity, urgency and the correct next action. The user may need a simple way to keep the details and ask another person. The project addresses this interaction problem through a small sequence of actions, rather than a complex automated detection system.

Objectives

- Provide a clear form for recording messages and calls.
- Highlight concerns such as pressure, secrecy and requests for private codes.
- Make known contacts accessible for voluntary calls, SMS and sharing.
- Preserve checks and outcomes for follow-up.
- Demonstrate the basic components in the Mobile App Development syllabus.

Scope

The application works on one device, with no login, backend or family synchronisation. Effectiveness in preventing scams has not been measured. No claim of production readiness or global novelty is made.

2. Requirements and modules

Development environment

The source project is intended for Android Studio with Java 17 compatibility. Its configuration sets minimum API 26, target API 36 and compile SDK 36.1. The existing build uses AGP 9.3.2 and Gradle 9.5.0. SDK availability and Gradle sync must be checked before execution.

Functional modules

• Home: start a check, see unresolved-check count, call a trusted person and open practice.

• Guided check: choose a source, enter a description and select five possible warning signs. Shared text can prefill the description.

• Check history: search messages/notes, filter outcomes and reverse date ordering.

• Case details: view the concern, edit a sharing preview, record an outcome/note and delete after confirmation.

• Trusted people: add/edit/remove contacts, store relationships, mark favourites, search and open calling/SMS apps.

• Learning: six awareness cards and a five-question offline practice quiz with feedback, scoring and retry.

• Reminders: date/time selection, one general follow-up, cancellation and a test notification.

Usability and persistence

The interface uses a restrained navy-and-teal palette, large controls and rounded cards. A larger-text preference is saved on the device. SQLite preserves records after the app closes. Blank concerns and invalid contact numbers are rejected; actual readability and behaviour still require device testing.

Permissions

The app requests notification permission where required and registers a boot receiver to restore the reminder. It does not request access to read SMS, monitor calls or read the system address book. No external UI library is required.

3. Architecture and workflow

Component responsibilities

MainActivity owns the toolbar, bottom tabs, navigation panel and reading settings. DashboardFragment constructs Home, Checks, People and Learn. BaseActivity and Ui provide shared screen construction and styling.

CheckActivity and DetailActivity manage the concern-to-outcome workflow. ContactDialog handles contact input and validation. CardAdapter binds records to custom rows. QuizActivity handles practice state. ReminderActivity and ReminderReceiver manage scheduling and notification delivery.

Data and service layers

Db extends SQLiteOpenHelper for local records. SharedPreferences stores text-size settings and reminder time. Intents connect to other screens and phone applications. AlarmManager schedules an inexact alarm; NotificationManager posts a local notification.

Typical workflow

- 1. Enter a concern or receive shared text from another app.
- 2. Choose the interaction source and relevant warning signs.
- 3. Validate the description and insert a case into SQLite.
- 4. Open the detail screen using the case ID in an explicit Intent.
- 5. Review sharing content or call someone already trusted.
- 6. Record Still unsure, Reported as suspicious or Verified independently.
- 7. Set a general follow-up reminder if needed.

Lifecycle and communication boundaries

The Fragment refreshes data when resumed and closes its database helper on destruction. Relevant form, quiz, filter and reminder state is preserved during recreation. The app cannot observe whether an external message was sent or a call answered, so it does not automatically change the case outcome after launching another app.

4. Database and implementation

SQLite schema

cases
_id — INTEGER PRIMARY KEY AUTOINCREMENT
message, flags, source, status, note — TEXT NOT NULL
created — INTEGER NOT NULL (timestamp)

contacts
_id — INTEGER PRIMARY KEY AUTOINCREMENT
name, phone, relationship — TEXT NOT NULL
favorite — INTEGER NOT NULL DEFAULT 0

These tables are independent. A case does not store its recipient and no foreign-key relationship connects cases and contacts. Reminder time and reading preferences are stored in SharedPreferences.

CRUD and ordering

ContentValues supplies inserted and updated values. Update and delete operations use selection arguments. Cursors are closed after reading. Contacts are ordered by favourite status and name; cases initially use descending creation time. The displayed list applies search, outcome filtering and optional reverse ordering.

Adapters and Intents

CardAdapter extends BaseAdapter. It inflates list_card.xml when a recycled view is unavailable, then binds the badge, title and subtitle. It supports custom ListView rows and the learning GridView. Explicit Intents pass a case ID or tab index. Implicit Intents support sharing, the dialer, SMS composition and browser guidance. Missing compatible apps produce a Toast.

Validation rules

Descriptions are required and limited to 5,000 characters; notes are limited to 2,000. Contact names are required. Phone numbers accept 6–15 digits with an optional leading plus after common separators are removed. The Verified independently outcome requires an explanatory note.

5. Interface and syllabus mapping

Interface design

Four bottom tabs keep main destinations visible. A toolbar button opens a left navigation panel. Options and submenu items expose additional actions and reading settings. Long-press context menus support record actions. Empty states explain how to begin, and the larger-text preference supplements system font scaling.

Unit 1 — Android foundations

The project demonstrates Java source classes, XML resources, a manifest, an application ID and Gradle configuration. Android Studio setup, SDK selection, running on an emulator or phone, and APK generation are part of the intended course demonstration. A successful APK build remains unverified.

Unit 2 — UI and lifecycle

- Views/layouts/drawables: forms, buttons, cards and XML screen/row layouts.
- ListView/GridView/adapters: repeated record rows and awareness cards.
- Menus: toolbar options, Help & settings submenu and context menus.
- Pickers/Spinner: date/time, concern source and case outcome.
- Activities/Fragments: navigation, resumed-data refresh and saved state.

Unit 3 — Communication and notifications

Explicit and implicit Intents demonstrate internal navigation and external app communication. Extras transfer identifiers between Activities. The navigation panel demonstrates a drawer using standard views. AlertDialog and Toast provide interaction feedback. NotificationManager, channels, PendingIntent and permission handling support local reminders.

Scope decisions

Remote push notifications are not implemented: there is no server or Firebase integration. A blocking ProgressDialog is not needed for this local workflow. Legacy platform Fragments keep the project dependency-free; a future production version could migrate to AndroidX.

6. Testing, privacy and limitations

Completed checks

• Java syntax parsing passed for 13 source files.

• Six XML files parsed; resource references and manifest component names were checked.

• SQLite schema creation and CRUD smoke checks passed.

• Favourite ordering, Unicode text and apostrophes were included in database checks.

Outstanding validation

APK compilation and lint were attempted but blocked before Java compilation because access to installed SDK metadata was denied. Running adb was also denied. Syntax checks do not establish Android type correctness, installation success, visual quality or runtime behaviour. No verified working APK or device-test result is claimed.

Pending acceptance tests

• Build, install and launch; create/update/delete records and restart the app.

• Verify search, filters, favourites, contact validation and missing external apps.

• Rotate during forms and the quiz; test large text and landscape layouts.

• Test notification permission granted/denied, disabled channels, replacement, cancellation and reboot restoration.

Privacy and operational limits

Records remain in private app storage unless voluntarily shared. Backups are disabled. The app does not read SMS, monitor calls or access the phone’s address book. No additional encryption layer is implemented, and uninstalling removes the data. Users are asked not to enter passwords, OTPs or account details.

There is one general reminder, not one per case. Android may delay delivery; force-stop or disabled notifications can prevent it. FamilyCheck does not synchronise family responses, file fraud reports, provide emergency support or guarantee recognition of scams.

7. Conclusion and future scope

Conclusion

FamilyCheck applies the foundations of Mobile App Development to a practical interaction problem. It gives users a structured sequence: pause, record a concern, notice warning signs, contact someone trusted and save the outcome. Local storage and existing phone apps keep the architecture manageable for a basic Java/XML project.

The documented implementation includes navigation, adapters, guided checks, contacts, practice and reminders. Source-level validation is complete; build and runtime validation remain necessary. Accessibility and practical benefit should be evaluated with real users after the application runs reliably.

Future improvements

- Complete compilation, lint and device testing first.
- Evaluate text sizes, labels and navigation with older adults.
- Add regional languages and optional text-to-speech.
- Introduce per-case reminders and reminder history.
- Consider user-controlled backup/export and AndroidX migration.

Classroom demonstration

Add a test contact and favourite it. Save a fictional pressure/secrecy message and review its details. Open and cancel the sharing chooser. Demonstrate search, filtering, a quiz question, larger text and a test notification. Avoid real private data and unintended messages or calls.

Project sources

1. FamilyCheck/README.md — scope, build instructions and verification status.
2. app/src/main/java/org/familycheck/senior/ — implementation.
3. AndroidManifest.xml and res/ — components and resources.
4. Mobile App Development Units 1–3 — supplied syllabus.

Prepared for Aravindh P and Richu James. No supervisor, approval, user-study result or successful runtime test has been invented.